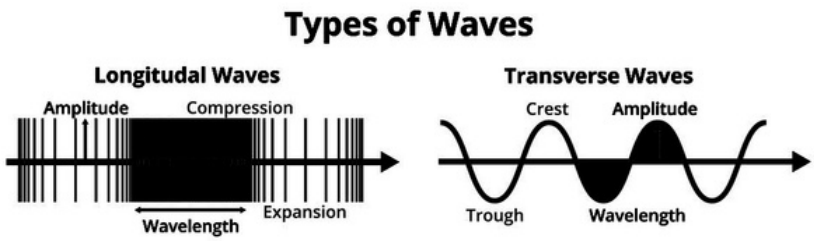
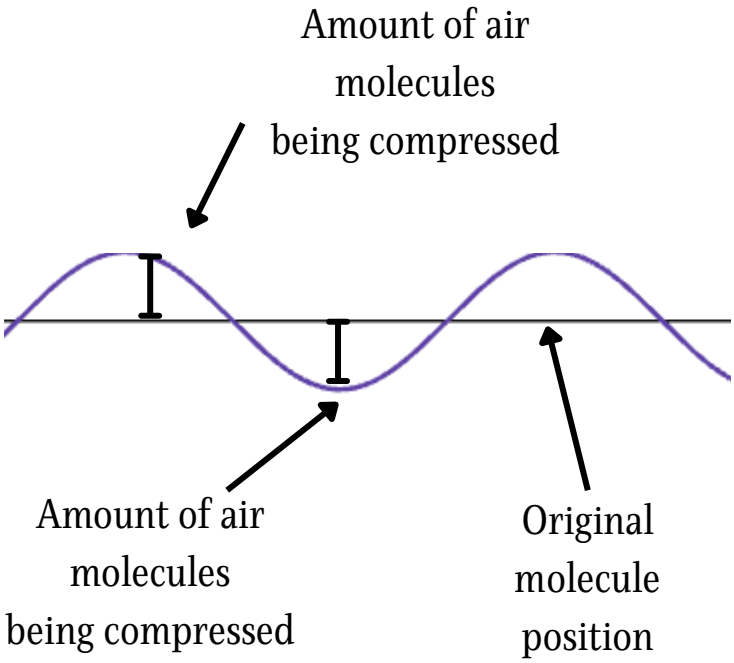
SOUND WAVES (KRISH)

While many believe that sound waves in the real world look like sine waves, that isn't actually the case. In fact, sound waves aren't transverse (oscillating perpendicular to its axis of motion, "up and down") but longitudinal (oscillating parallel to its axis of motion, "compressing and expanding"). So if that's the case, how can sine waves be represented by sinusoidal functions?





Sound waves are longitudinal because sound essentially represents pressure oscillations of air molecules (the compressing and releasing). Therefore, we can graph these oscillations in the form of a sinusoidal function, where a positive value represents compressing molecules together, while a negative value represents pushing molecules apart.

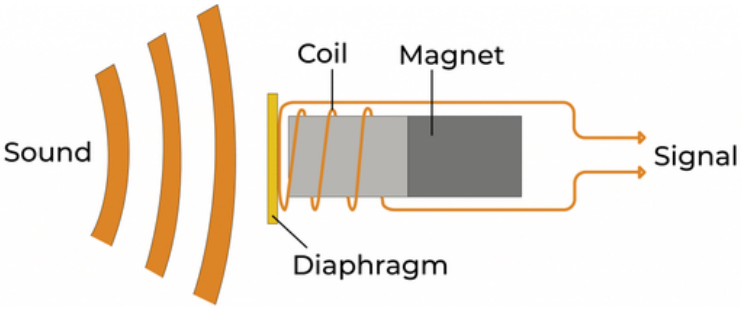
Seems pretty simple, right? But there's a caveat. Recall that the sine wave that we represented the sound with doesn't actually represent a range of sounds being made. Instead, one wave only contributes to one pure tone (a simple sound). When we consider the sounds that we hear in our everyday lives, from music to even our voices, we must realize that these sounds are actually made of many different sound waves (called overtones) all combined together to produce a complex sound. So if we want to create an equation for the music that comes from our vinyl, we need to find the simple components (pure tones) of the sound it makes, find their equations, and then combine them all to get our final function.

Without further ado, let's get to the fun part: building the function!

BUILDING THE FUNCTION

Before we figure out what composes our function, we first need to have... an actual function. So how exactly do we get our function?

When we record sounds, what actually happens is that a microphone turns the vibrations of air molecules into electrical voltage. It does so using a diaphragm (a thin membrane) attached to a coil with a magnet. When sound hits the diaphragm, it vibrates, moving the coil, and producing electrical voltage (by Faraday's Law). Different sound waves will cause the diaphragm to move differently, allowing us to interpret them as different signals.



Before computers can use these signals, they go into an ADC (Analog to Digital Converter like a sound card or audio interface), which will measure the voltage at fixed time intervals (sampling) and convert each signal into a number. Once that's done, the computer can now actually use the data, meaning we have a function!

However, this function isn't a continuous function, but a discrete function. That's because computers can't physically read a continuous stream of values, so it must read them in intervals, leading to discrete points. So instead of **x(t)**, where **x** is our function and **t** is time in seconds, we get **x[n]**, where **n** is an integer index that allows us to access discrete points in our function **x**. It's important to keep this in mind because this function will have to be an approximation, since it is physically impossible to create an equation this way that perfectly represents a sound.

But now that we have our function, our next step is to find what it's actually composed of. To do so, we must perform a DFT (Discrete Fourier Transform), which will tell us what frequencies are actually present in our sound. The formula for a DFT is

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-i2\pi kn/N}$$

Where:
x[n] is the input signal (our function)
N is the total number of samples
k is the frequency bin index
X[k] is the complex output

This is essentially just a fancy way of saying “How much does **x[n]** look like the wave present at some frequency **k**?” In other words, it computes the dot product between **x[n]** and the complex sine wave representation of the wave at **k**. Our resulting function **X[k]** stores these complex values for each frequency **k**. If a frequency matches with **x[n]**, the result will usually be a huge sum. But if it doesn’t, then the result will usually be around 0.

When performing a DFT using a computer, it’s actually quite costly, being an $O(n^2)$ operation. So instead, we usually use an FFT (Fast Fourier Transform), which splits the function and recursively evaluates dot products (resulting in a time complexity of $O(n \log n)$).

You might also be wondering why we’re using complex numbers. We do so because we want to measure how **x[n]** compares to a sine AND cosine wave at frequency **k** so that we can analyze the wave at any given starting point. Euler’s formula states that

$$e^{i\theta} = \cos(\theta) + i\sin(\theta)$$

Allowing us to contain a sine and cosine component in one compact “object” (a complex rotating vector), rather than having to store 2 amplitude coefficients per frequency **k**. To actually get the amplitude and frequency values, we can convert this from Cartesian form to polar form. For instance, take any arbitrary complex value **a**+**bi**. To calculate the amplitude, you can use

$$A = \sqrt{a^2 + b^2}$$

and for the phase shift, we can use

$$\theta = \arctan(\frac{b}{a})$$

We can use this logic for all of our frequencies, which we will then combine to form our final equation. Mathematically, this would be represented as

$$x(t) \approx \sum_{k=0}^{N/2} A_k \cos(2\pi f_k t + \theta_k)$$

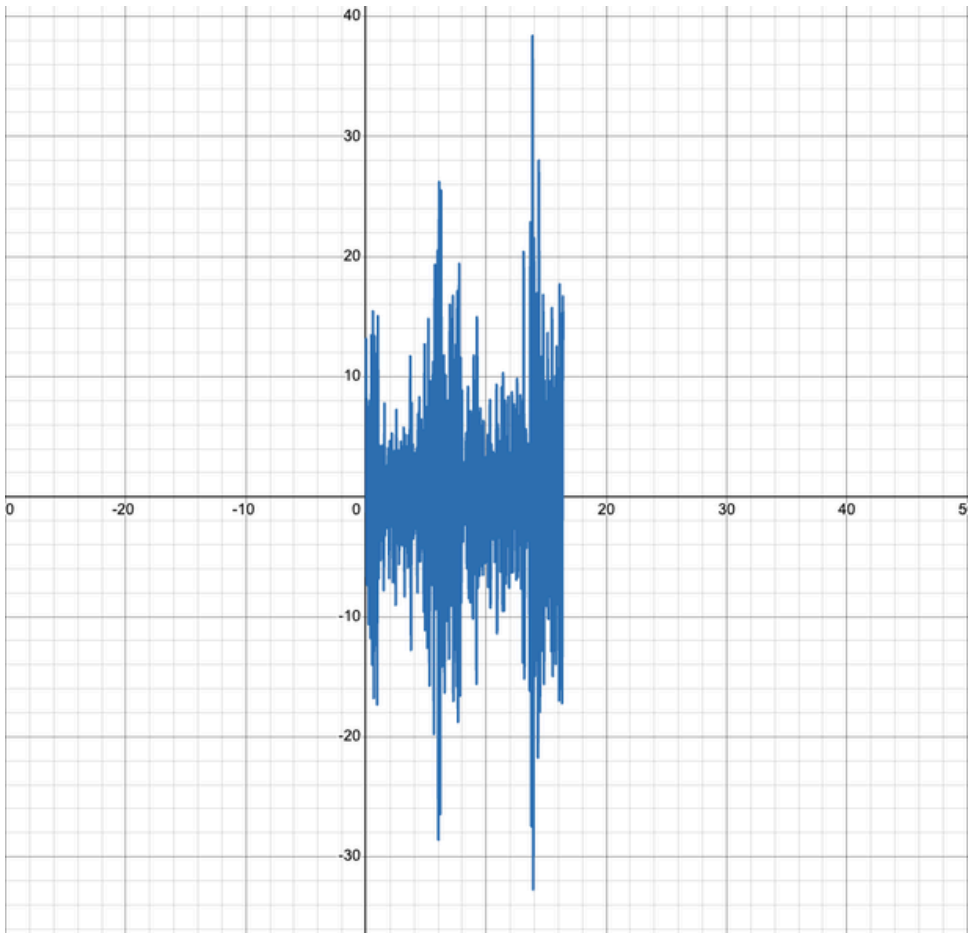
Where **x(t)** is our final function, **k** is the frequency index, $\mathbf{A_k}$ is the amplitude of the wave at **k**, $\mathbf{f_k}$ is the frequency of the wave at **k**, and θ_k is the phase shift of the wave at **k**. (Note that we use N/2 because we want to exclude negative frequencies)
And that’s our equation!... But did we really do all of that work JUST for an approximation?

The truth is, it is impossible for us to make a function that perfectly represents a complex sound. It was mentioned before that the function we've been analyzing is discrete, and as a result, we don't have a continous time axis. So when we try to equate this to **x(t)**, a continuous function, we must realize that it's really just an approximation.

On top of that, this is the BEST case scenario for our approximation, assuming that we would take the time to write out the hundreds of thousands of waves that make these complex sounds... In reality, we often just take the most significant sounds (waves with highest amplitudes) and use those. But even that is somewhat flawed, since computers often represent spikes of a frequency as a different frequency altogether, clouding the "highest amplitudes". At the end of the day, even an approximation of a complex sound takes huge amounts of complex math...

But that doesn't mean we're left completely empty-handed. I wrote a program to do all of this complicated math, and I found that with just a 16 second clip of a Taylor Swift song and the 20 most significant components, there were some pretty interesting results.

The graph I got ended up looking like this:



In this case, the x-axis represents **t** (time in seconds), and the y-axis represents **X(t)** (the air molecule vibrations, scaled down by a factor of 100). The domain has also been restricted to emphasize that this is a 16 second clip. Specifically,

$$x \in [0, 16.404]$$

If you look at it, it actually looks very similar to those visualizations you might see on a recording app. I also did some research on other applications for this, and I found that using this math and a different process, you can actually make things like sound visualizers for music videos.

Oh, and here's what our final approximation looked like as an actual equation with everything rounded to 3 decimal places:

$$2.201\cos(2\pi 733.053x - 1.459) + 1.941\cos(2\pi 734.760x + 1.575) + 1.939\cos(2\pi 735.979x + 3.068)$$
$$+3.310\cos(2\pi 736.101x - 2.295) + 3.718\cos(2\pi 736.223x - 1.365) + 3.235\cos(2\pi 736.711x + 0.247)$$
$$+2.821\cos(2\pi 736.832x + 1.279) + 2.569\cos(2\pi 737.015x - 0.984) + 3.008\cos(2\pi 737.076x - 0.360)$$
$$+2.051\cos(2\pi 737.137x + 0.646) + 2.391\cos(2\pi 737.381x - 1.496) + 2.050\cos(2\pi 737.503x - 0.246)$$
$$+2.288\cos(2\pi 738.174x - 0.686) + 1.926\cos(2\pi 738.296x + 1.134) + 2.483\cos(2\pi 738.417x - 3.084)$$
$$+2.564\cos(2\pi 738.539x - 1.324) + 2.661\cos(2\pi 738.661x + 0.171) + 2.346\cos(2\pi 738.783x + 1.946)$$
$$+2.423\cos(2\pi 740.185x + 0.741) + 0.004\cos(2\pi 1.158x - 0.132)$$

Yeah… it's probably good that instead of using 100 waves like originally planned, I chose to use just 20… But what can we actually gather from this?

For starters, we can answer the question: how are amplitude and frequency significant in approximating these functions? We've seen that frequencies can determine how components are categorized in our DFT, and we've seen that amplitude plays a role in the actual approximation (measuring significance). Furthermore, we also got to see that even then, using amplitude without considering the limitations of the microphone results in our approximation not being completely optimal.

Another thing we can answer is: how is sound actually interpreted by a computer? We've seen that it utilizes a microphone which detects the vibrations of air molecules and converts that into electrical signals which are then processed by the computer.

Additionally, we can also answer the question: what's the difference between stereo and mono sound files, and how does that affect the DFT? We found that DFTs actually require a mono sound file to work (since it expects only one channel of sound) so you need to convert any stereo sound files into mono by averaging the two channels. That's also why stereo can be converted to mono, and not vice versa.

Finally, we can also answer: what is lost when representing sound with sinusoidal functions? As we've seen, many of the nuances of the actual sound were not represented in our approximation, since we didn't use all of the components, and we only had discrete points to work with in the first place. Essentially, an approximation was basically inevitable even from the start.

Overall, this was an interesting project that connected computers, physics, and math into one exploration, and while it was quite difficult, it was definitely fun to work on and research!

And one more thing… if you're interested in the code I used to write this, it is displayed on the next page AND attached for your convenience (if you're looking at my submission)!

main.py

Run the program from this file using the **python** (or **python3**) command

You can customize your approximation from the program file. Also remember to use **.wav** files!

```python
from dependencies import *

NUM_WAVES = 20 # Specifies the number of waves to approximate with
SCALE = 100 # Scale down factor
DESTINATION = "equations.txt"  # Destination file for equations
LATEX_MODE = False  # True outputs out LaTeX formatting, False outputs Desmos formatting

# Prints your configurations
if LATEX_MODE:
    print("Mode: LATEX")
else:
    print("Mode: DESMOS")

print(f"Scale: {SCALE}")
print(f"Wave Count: {NUM_WAVES}")
print(f"Destination: {DESTINATION}")

# Gets all filenames to get the equation for
filenames = []
while True:
    filename = input("Enter in the next filename (type QUIT to finish): ").strip()

    if filename == "QUIT":
        break

    if file_exists(filename):
        filenames.append(filename)
    else:
        print(f"Error: File {filename} not found!")

ensure_file(DESTINATION) # Ensures the destination file exists
write_all_equations(filenames, DESTINATION, NUM_WAVES, SCALE, LATEX_MODE)

# Prints a message if it succeeds
N = len(filenames)
if N > 0:
    print(f"Wrote all {N} equations to {DESTINATION}.")
```

dependencies.py

(Note: For this file, you need to have the **scipy** and **numpy** libraries installed on your machine. Make sure you have **pip** downloaded, so you can install the libraries easily)

```python
from scipy.io import wavfile
import numpy as np

def write_all_equations(filenames, destination, num_waves, scale, latex_mode):
    for filename in filenames:
        write_equations(filename, destination, num_waves, scale, latex_mode)

# audio file, destination file
def write_equations(filename, destination, num_waves, scale, latex_mode):
    # Fs, x[n]
    rate, data = wavfile.read(filename)

    # Converts to mono if needed by averaging the columns
    if data.ndim == 2:
        data = data.mean(axis=1)

    # X
    transform = np.fft.fft(data)

    # with /len(transform), it makes the amplitudes actually useful amp values
    amps = np.abs(transform) / len(transform)
    phases = np.angle(transform)
    freqs = np.fft.fftfreq(len(data), 1/rate)

    N = len(transform)

    # Cut out negatives
    amps = amps[:N//2]
    freqs = freqs[:N//2]
    phases = phases[:N//2]

    # Get top indices
    top_indices = np.argsort(amps)[-num_waves:]
    top_indices = sorted(top_indices, key=lambda i: freqs[i])

    #Scaled down amplitudes, writes the equation
    with open(destination, 'a') as file:
        file.write(f"{filename}\n")
        for j in range(num_waves-1):
            i = top_indices[j] # Gets index from top indices
            write_equation(file, amps[i]/scale, freqs[i], phases[i], False, latex_mode)
```

```
        write_equation(file, amps[num_waves-1]/scale, freqs[num_waves-1], phases[num_waves-1], True, latex_mode)

        if not latex_mode:
            file.write("\\left\\{0\\le x\\le " + f"{N/rate:.3f}" + "\\right\\}\n")

def write_equation(file, amp, freq, phase, is_end, is_latex):
    if is_latex:
        file.write(f"{amp:.3f}\\cos(2\\pi {freq:.3f} x")
        # Handles negatives
        if phase > 0:
            file.write(f" + {phase:.3f})")
        else:
            file.write(f" - {abs(phase):.3f})")

        # Handles spacing (prints out each equation line by line)
        if not is_end:
            file.write(" +\n")
    else:
        file.write(f"{amp:.3f}cos(2\\pi{freq:.3f}x")

        # Handles negatives
        if phase > 0:
            file.write(f" + {phase:.3f})")
        else:
            file.write(f" - {abs(phase):.3f})")

        # Handles spacing (prints all equations on one line)
        if is_end:
            file.write("\n")
        else:
            file.write(" + ")

# Ensures that the file exists and is blank
def ensure_file(filename):
    with open(filename, 'w'):
        pass

# Returns whether a file exists
def file_exists(filename):
    try:
        with open(filename, 'r'):
            return True
    except FileNotFoundError:
        return False
```